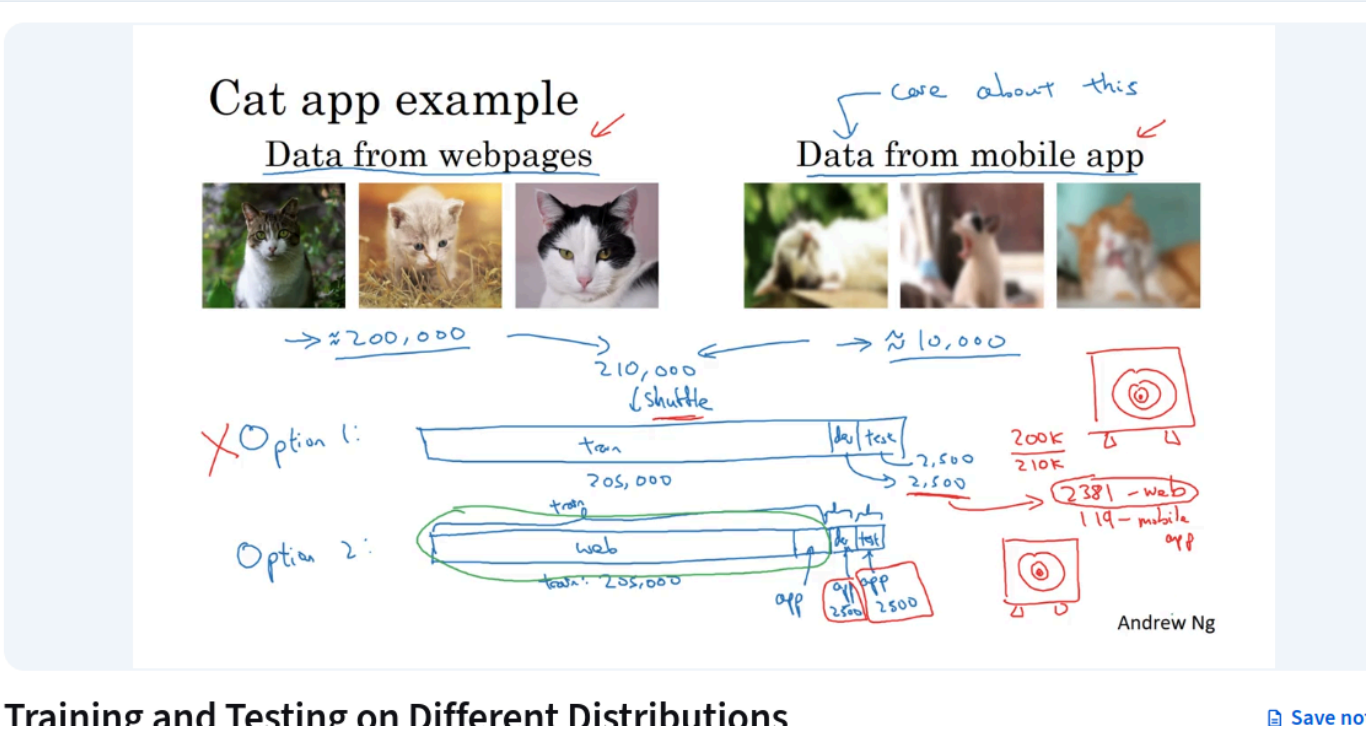
Mismatched Training and Dev/Test Set

Training and Testing on Different Distributions



Training and Testing on Different Distributions

Save no

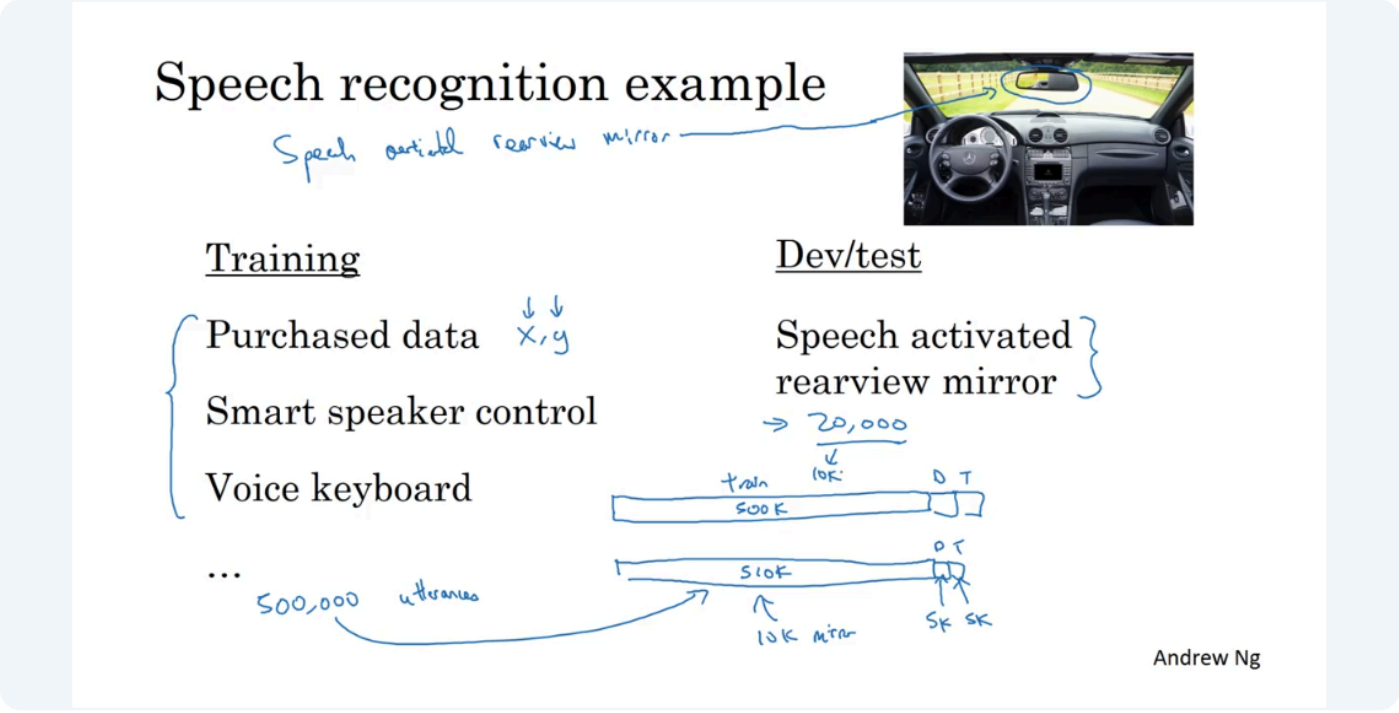
Here's a **concise summary of the key points**:

- **Deep learning needs lots of data**, so teams often add any data they can find—even if it comes from a **different distribution** than what they actually care about.
- **Problem setup example**
  - Goal: classify cat photos uploaded by **mobile app users** (real target distribution).
  - Data available:
    - **10k mobile app images** (blurry, amateur, realistic)
    - **200k web images** (high-quality, professional, different distribution)
- **Bad option: mix everything and randomly split**
  - Combine all data, then randomly create train/dev/test.
  - Result: dev/test sets are **dominated by web images**.
  - Consequence: you optimize for the **wrong distribution**.
  - Dev set no longer reflects what users actually care about.
- **Good option (recommended)**
  - **Training set**: mostly web images + some mobile images

    (e.g., 200k web + 5k mobile)
  - **Dev set**: only mobile images
  - **Test set**: only mobile images
- **Why this works**
  - Dev/test sets define the **target you're aiming for**.
  - Keeping dev/test aligned with the real-world distribution ensures:
    - Correct model comparison
    - Optimization for what actually matters
- **Trade-off**
  - Training data distribution ≠ dev/test distribution
  - This is acceptable and often **better in the long run**

- There are techniques to handle this mismatch (discussed later)

- **Core takeaway**
  - **Dev and test sets must match the real deployment distribution**
  - It’s okay—and often smart—for the training set to be different if it gives you more data



Here’s a **clear key-point summary** of this example:

- **Scenario**
  - Product: **speech-activated rearview mirror** for cars.
  - Target use: navigation queries (street names, addresses).
  - Target data distribution is **very specific** and different from generic speech data.
- **Available data**
  - Large amount of **generic speech recognition data** (e.g., smart speakers, voice keyboards): ~500k utterances (lời nói).
  - Smaller amount of **rearview-mirror–specific data**: ~20k utterances.
- **Recommended data split**
  - **Training set**:
    - Use *all* generic speech data.
    - Optionally include some rearview-mirror data.
  - **Dev & test sets**:
    - Use **only rearview-mirror data**, since that’s what the product must perform well on.
- **Two reasonable splits**
  1. Train: 500k generic

     Dev: 10k rearview mirror

     Test: 10k rearview mirror
  2. Train: 500k generic + 10k rearview mirror

     Dev: 5k rearview mirror

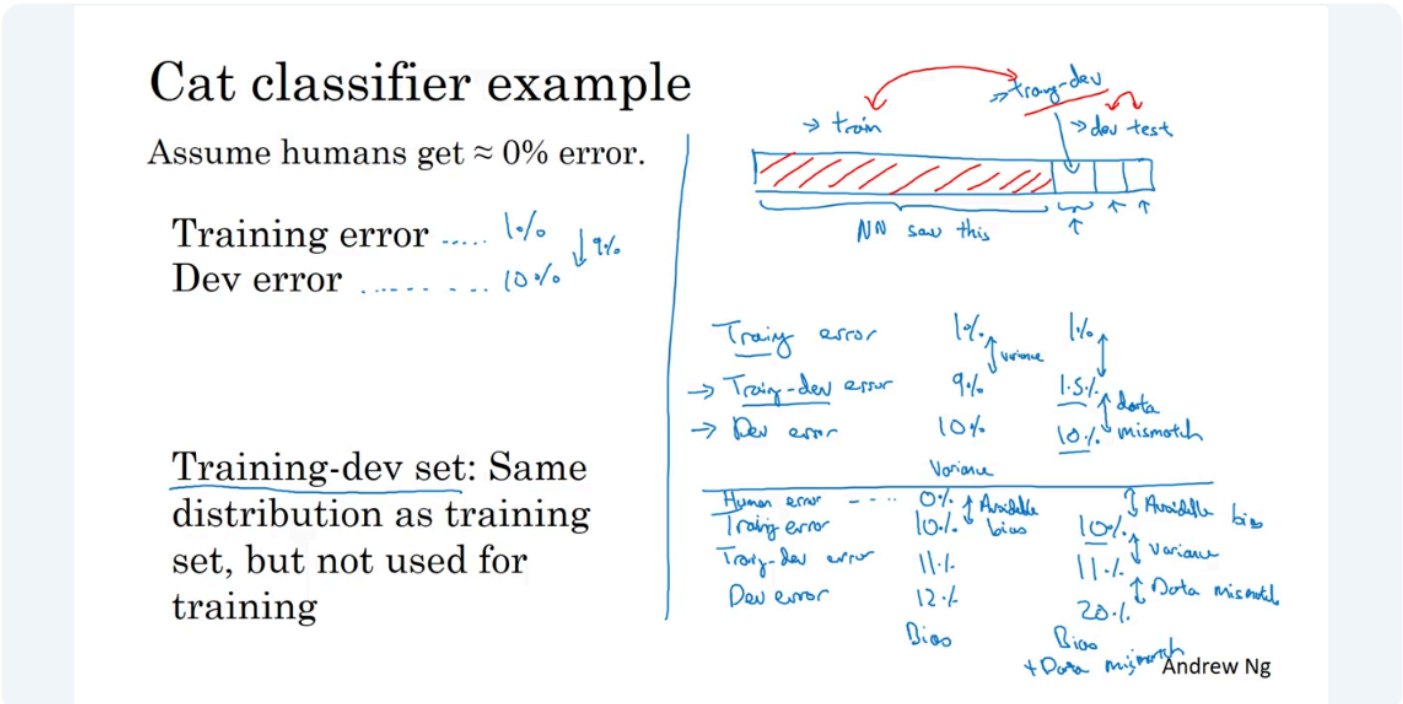
     Test: 5k rearview mirror
- **Why this works**
  - Dev/test sets define the **true target distribution**.
  - Training on additional out-of-distribution data gives you **far more training signal**.
  - This often leads to **better overall performance**, despite distribution mismatch.

- **Key principle**
  - It’s okay—and often beneficial—for the **training set distribution to differ** from dev/test.
  - What matters most is that **dev and test match the real-world usage**.
- **Open question raised**
  - Should you *always* use all available data?
  - Answer: **Not always** (to be discussed next).

👉 **Core takeaway:**

Using large amounts of non-target data in training is often helpful, as long as dev and test sets reflect the real deployment scenario.

Bias and Variance with Mismatched Data Distributions



Here’s a **clear, structured summary of the key ideas** about bias/variance when **training and dev distributions differ**:

Why standard bias/variance analysis breaks

- When **training and dev data come from different distributions**, a gap between training error and dev error is **ambiguous**.
- The error increase could be due to:
  1. **Variance** (model doesn’t generalize to unseen data)
  2. **Data mismatch** (dev data is inherently different/harder)
- Since *two things change at once*, you can’t diagnose the problem correctly.

Solution: introduce a training-dev set

- **Training-dev set**:
  - Same distribution as the training set
  - Not used for training
- Purpose: isolate **generalization error** from **distribution mismatch**

Dataset roles

Set	Distribution	Used for training?
Training	Training distribution	✅
Training-dev	Training distribution	❌

Set	Distribution	Used for training?
Dev	Target (real-world) distribution	❌
Test	Target distribution	❌

How to interpret errors

1. High variance

- Training error: **low**
- Training-dev error: **high**
- Dev error: similar to training-dev

  ➡️ Model doesn't generalize even to unseen data from the *same distribution*

Example

- Train: 1%
- Train-dev: 9%
- Dev: 10%

  → **Variance problem**

---

2. Data mismatch

- Training error: low
- Training-dev error: low
- Dev error: high

  ➡️ Model generalizes well, but to the *wrong distribution*

Example

- Train: 1%
- Train-dev: 1.5%
- Dev: 10%

  → **Data mismatch problem**

---

3. High bias (avoidable bias)

- Training error is far above human/Bayes error

  ➡️ Model is too simple or underfitting

Example

- Train: 10%
- Train-dev: 11%
- Dev: 12%

  → **High bias**

---

4. High bias + data mismatch

- Training error: high
- Training-dev error: slightly higher
- Dev error: much higher

Example

- Train: 10%
- Train-dev: 11%

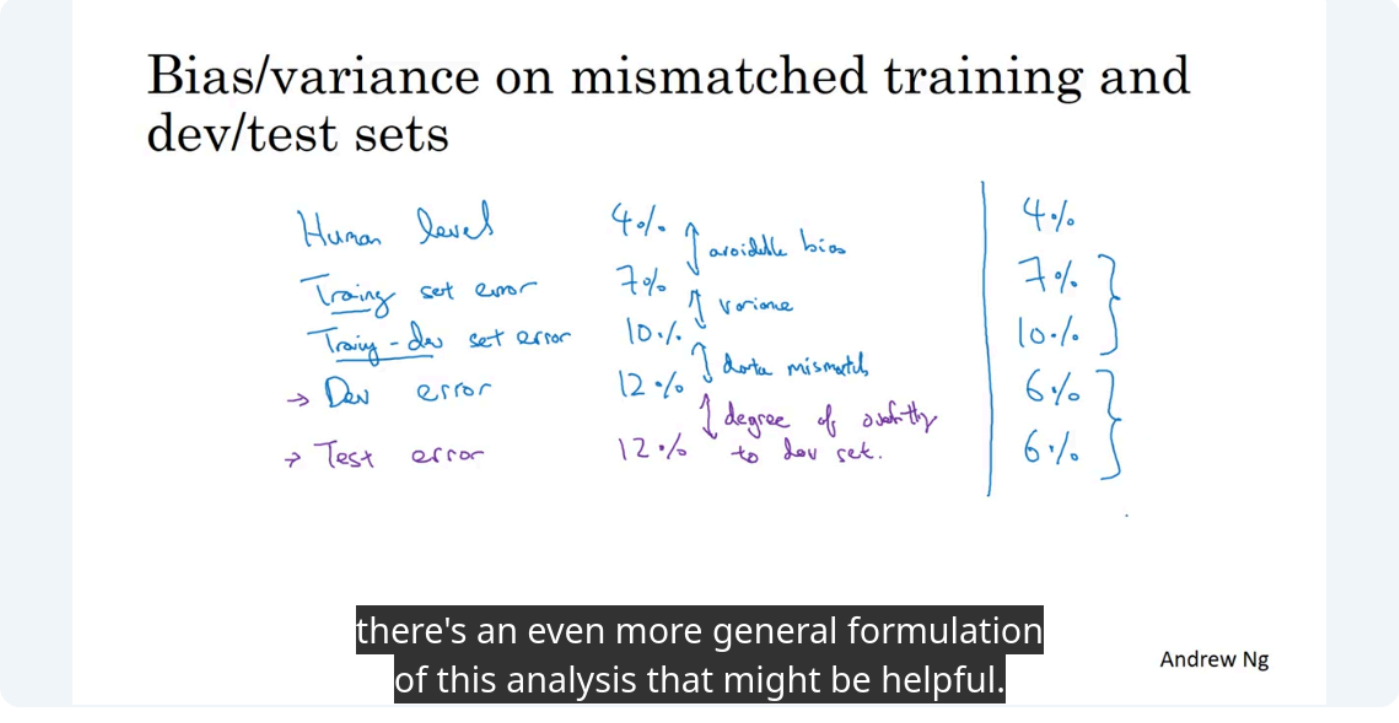
- Dev: 20%

  → **Bias + data mismatch**

Key takeaways

- When distributions differ, **training vs dev error alone is misleading**
- The **training-dev set** lets you:
  - Diagnose **variance**
  - Separate it from **data mismatch**
- Correct diagnosis → correct prioritization:
  - Bias → better model / features
  - Variance → regularization / more data
  - Data mismatch → better-matched training data

👉 **Core insight:**
You need a **training-dev set** to do meaningful bias/variance analysis when training and dev data come from different distributions.



Here's a **clean summary of the general principles** for diagnosing problems when **training, training-dev, dev, and test sets** are involved:

Key errors to track

1. **Human-level error** (proxy for Bayes error)
2. **Training error**
3. **Training-dev error**

   (same distribution as training, but unseen during training)
4. **Dev error**
5. *(Optional)* **Test error**

What each gap tells you

1. Avoidable bias

- **Human-level error → Training error**
- If training error ≫ human error → **high bias / underfitting**

- Focus: bigger models, better features, longer training

---

2. Variance

- **Training error → Training-dev error**
- Measures generalization to unseen data from the *same distribution*
- Large gap → **high variance**
- Focus: regularization, more training data, data augmentation

---

3. Data mismatch

- **Training-dev error → Dev error**
- Measures mismatch between training distribution and target distribution
- Large gap → **data mismatch problem**
- Focus: better-matched training data, domain adaptation

---

4. Dev overfitting

- **Dev error → Test error**
- Dev and test are same distribution
- Large gap → **overfitting to the dev set**
- Fix: get a larger dev set or reduce tuning on dev

Example interpretation

Human: 4%Train: 7%Train-dev: 10%Dev: 12%

- 4% → 7% → **avoidable bias**
- 7% → 10% → **variance**
- 10% → 12% → **data mismatch**

Important caveat

- Errors **don't have to monotonically increase**
- Sometimes dev/test data is **easier than training data**
  - e.g., training data is noisier or harder
- In such cases, dev error can be **lower** than training-dev error

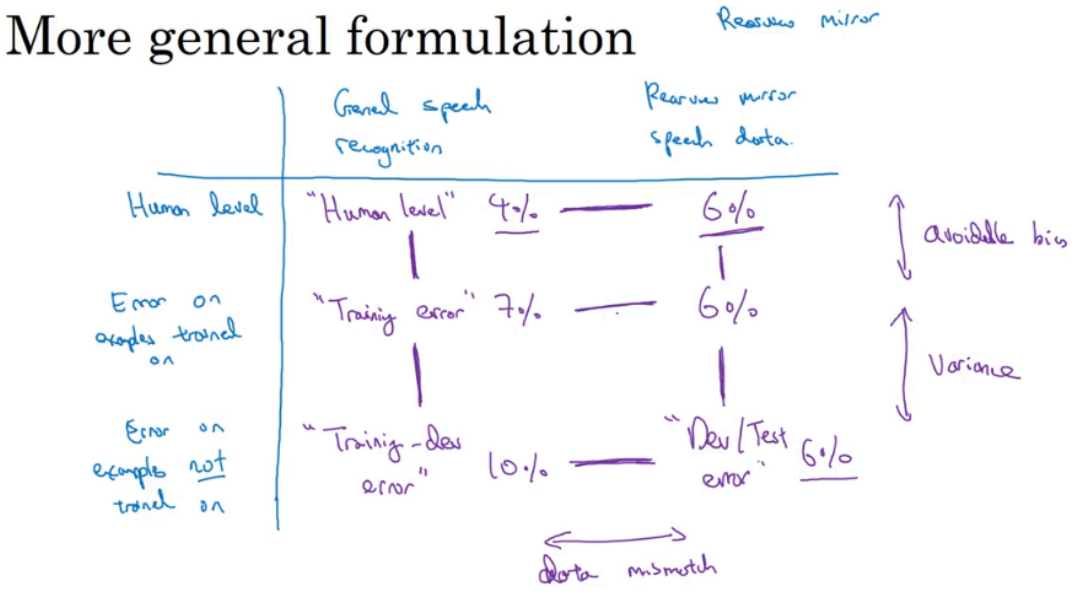
---

Big picture

- These error comparisons give you a **diagnostic tool**, not strict rules
- The goal is to understand:
  - *Where performance is being lost*
  - *What to fix next*

👉 **Core takeaway:**

By comparing **human → train → train-dev → dev → test errors**, you can systematically identify whether your main bottleneck is **bias, variance, data mismatch, or dev-set overfitting**, and prioritize work accordingly.



Alright, let me distill all of that into **clear, general principles** you can actually *use* when debugging a ML system. I'll keep the structure tight and practical.

---

1. The core idea: compare errors across *data* and *experience*

You are analyzing performance along **two dimensions**:

1. **Which data distribution?**
   - Training distribution (e.g. general speech data)
   - Target distribution (e.g. rear-view mirror in-car speech)
2. **Has the model seen the example during training?**
   - Seen (trained on)
   - Unseen (not trained on)

Humans give you a **reference floor** for what's achievable.

---

2. The 5 key quantities (and what each tells you)

Let's define them clearly:

1. **Human-level error (on a dataset)**
   → Proxy for Bayes / irreducible error
   → Sets a realistic lower bound
2. **Training error**
   → Error on examples the model trained on
   → Measures how well the model fits the training data
3. **Training-dev error**
   → Same distribution as training data, but unseen by the model
   → Measures *generalization within the same distribution*
4. **Dev error**
   → Unseen data from the target distribution
   → Measures performance on what you actually care about
5. **Test error**
   → Also target distribution, untouched until the end

→ Measures dev-set overfitting

---

3. How to interpret the gaps (this is the money part)

Assume this example:

- Human-level error: **4%**
- Training error: **7%**
- Training-dev error: **10%**
- Dev error: **12%**

(A) Avoidable bias

Training error – Human error

```
7% – 4% = 3%
```

➡️ Your model is **underfitting**

➡️ Focus on:

- Bigger model
- Better architecture
- Better optimization
- More training time

(B) Variance

Training-dev error – Training error

```
10% – 7% = 3%
```

➡️ Model fits training data but doesn't generalize well

➡️ Focus on:

- More training data
- Regularization
- Data augmentation

(C) Data mismatch

Dev error – Training-dev error

```
12% – 10% = 2%
```

➡️ Training and dev data come from *different distributions*

➡️ This error **cannot** be fixed by just reducing bias or variance

---

(D) Dev set overfitting (optional but important)

Test error – Dev error

```
Large gap ⇒ you overtuned the dev set
```

➡️ Solution:

- Increase dev set size
- Freeze dev set earlier

- Stop iterating on dev metrics

---

4. Why errors don’t always increase monotonically

Sometimes you see this:

- Human (train dist): 4%
- Training error: 7%
- Training-dev error: 10%
- Dev error: **6%** ← lower?!

This happens when:

- **Target data is easier** than training data
- Example: clean in-car speech vs noisy crowd-sourced speech

➡️ This is *not a bug*, just a distribution difference.

---

5. The full table formulation (general framework)

	Training distribution	Target distribution
Human error	How well humans do	How well humans do
Model (seen)	Training error	(optional)
Model (unseen)	Training-dev error	Dev / Test error

What this buys you:

- Lets you see **whether humans also find one distribution harder**
- Tells you if the model is:
  - Worse than humans
  - Matching humans
  - Limited by data quality rather than algorithms

---

6. Key insight from the rear-view mirror example

If:

- Human error (target) ≈ Model error (target)

➡️ You’re **already at human-level performance**

➡️ Further gains are hard and may require:

- Better sensors
- Better labels
- Problem reformulation

Not more model tweaks.

---

7. About fixing data mismatch (hard truth)

Unlike bias and variance:

- ❌ No clean, systematic recipe
- ❌ Regularization alone won’t fix it

Things that *may* help:

- Collect more data from the target distribution
- Data synthesis / augmentation to mimic target data

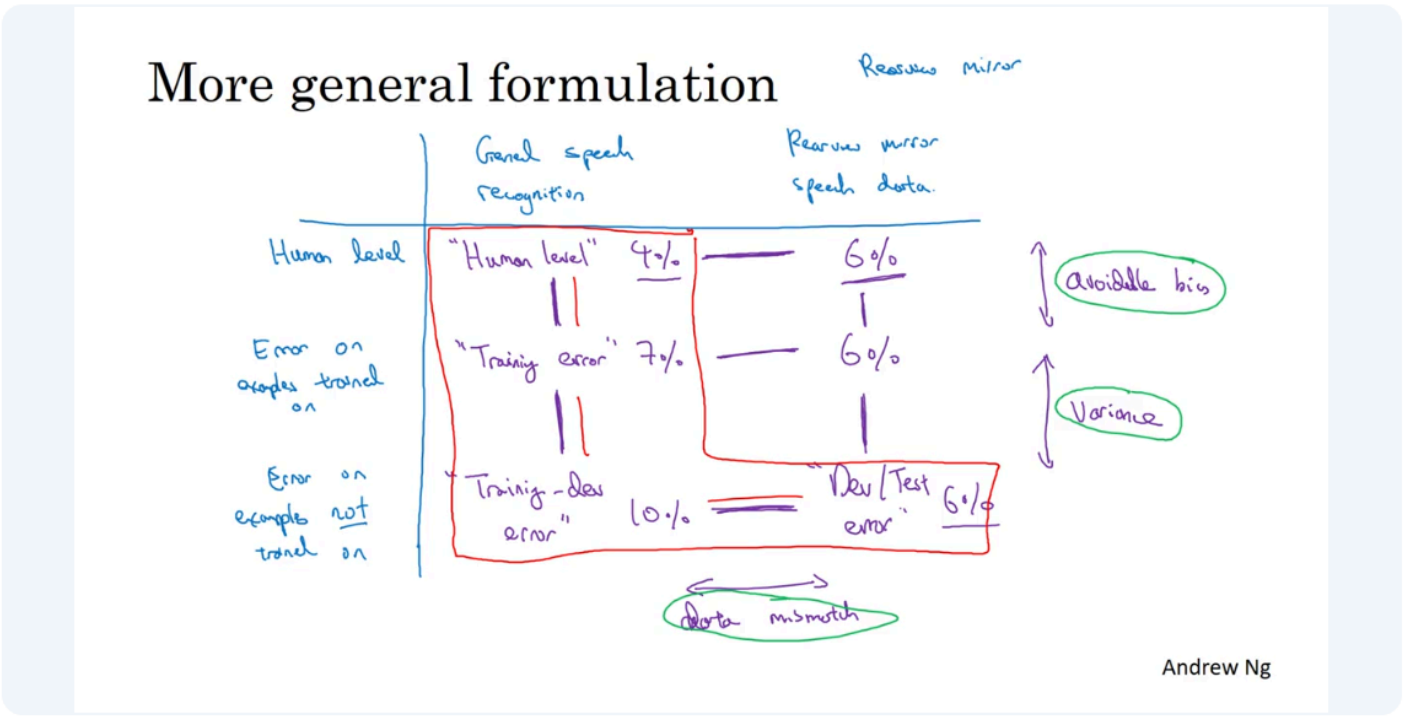
- Reweighting or sampling strategies
- Domain adaptation techniques

But error analysis is **essential** before trying anything.

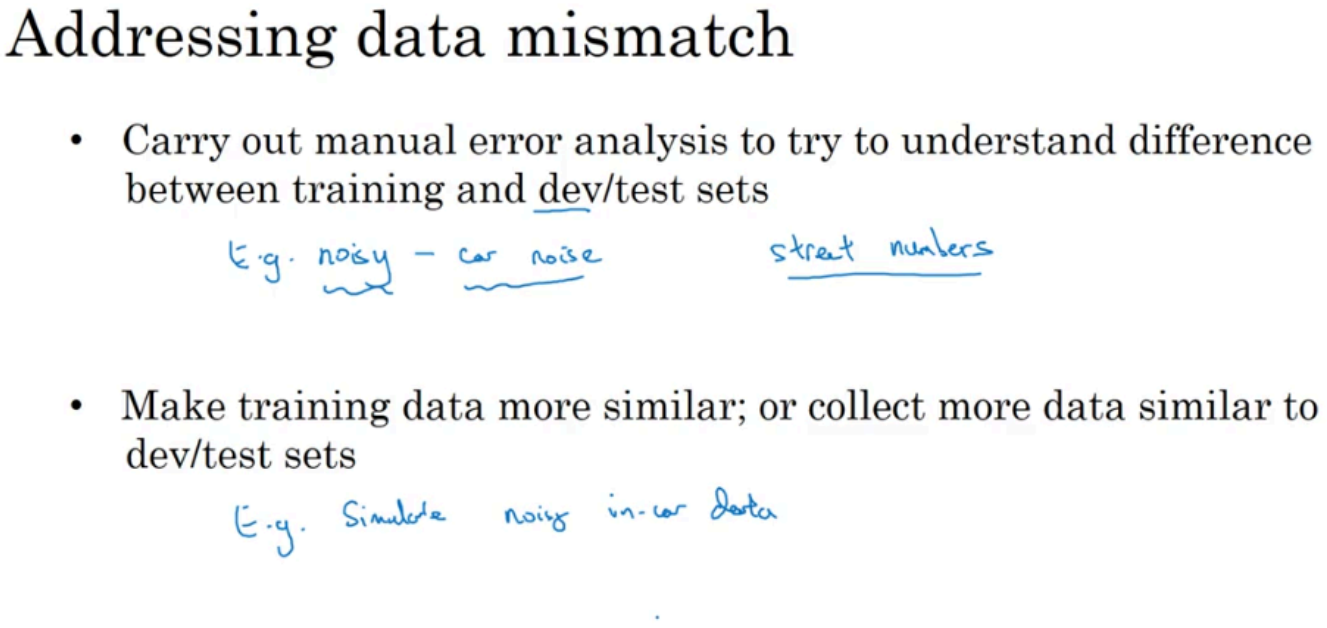
---

8. One-sentence takeaway

> By comparing human error, training, training-dev, dev, and test errors across distributions, you can precisely diagnose whether your problem is bias, variance, data mismatch, or dev overfitting—and avoid wasting time fixing the wrong thing.

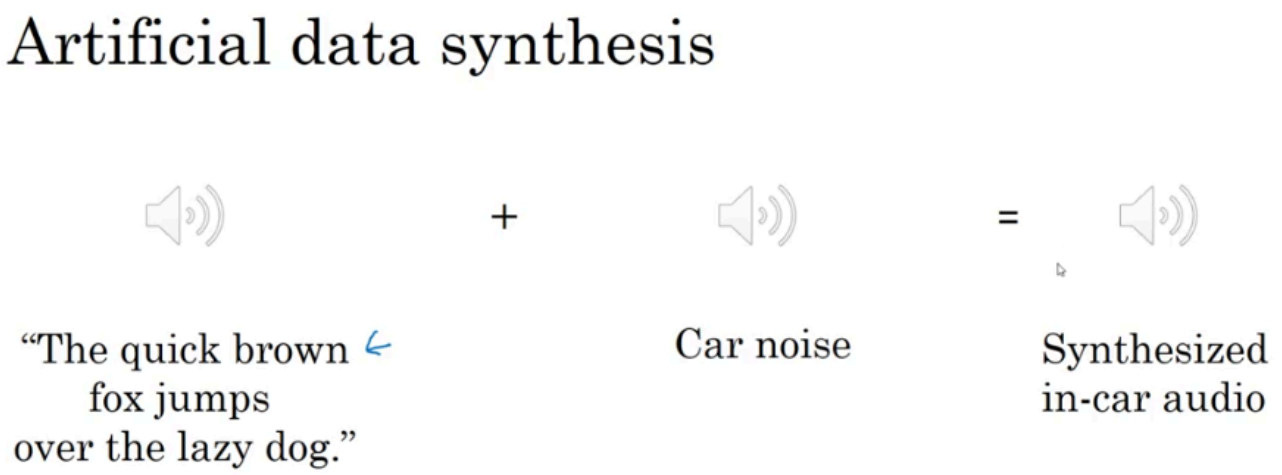


Addressing Data Mismatch



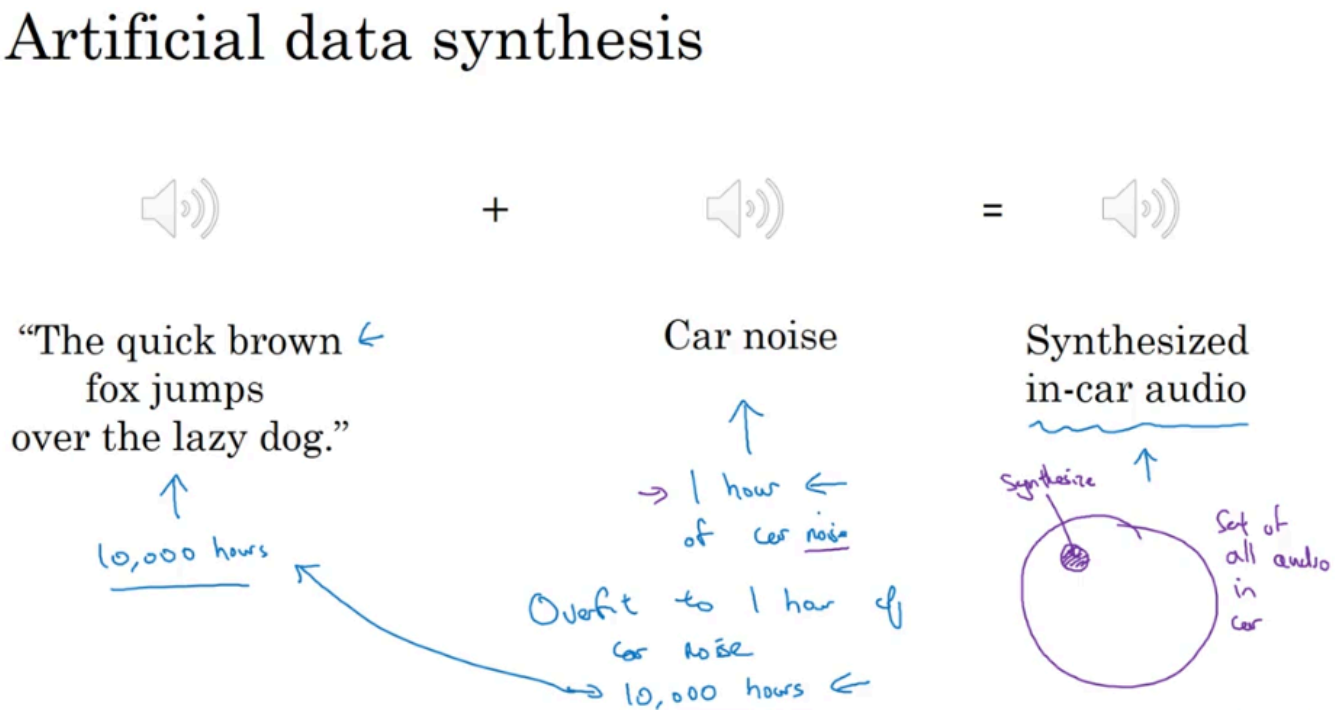
If your goal is to make the training data more similar to your dev set, what are some things you can do?

→ Artificial data systhesis



you have 10 000 hours of data that was recorded against a quiet background.

So, one thing you could try is take this one hour of car noise and repeat it 10,000 times in order to add to this 10,000 hours of data recorded against a quiet background. If you do that, the audio will sound perfectly fine to the human ear, but there is a chance, there is a risk that your learning algorithm will overfit to the one hour of car noise.



Artificial data synthesis (e.g., adding reverberation or car noise) can help quickly generate large amounts of training data without expensive real-world collection. This is useful when error analysis shows your model needs data that mimics specific environments, like audio recorded inside a car.

However, there’s a key risk: **lack of diversity leading to overfitting**. If you only have a small sample (e.g., 1 hour of car noise) and reuse it to create thousands of hours of data, the audio may sound fine to humans, but the model may overfit to that limited variation. This happens because the synthesized data covers only a tiny subset of all possible real-world conditions.

Takeaway: Artificial data is helpful, but it must be diverse. Using many unique samples (e.g., different car noises) is better than repeatedly reusing a small dataset.

Here’s a **simple, note-friendly version** you can use:

Artificial Data Synthesis (Example: Self-Driving Cars)

- When building a self-driving car, we need data to detect cars (e.g., draw bounding boxes).
- One idea: use **computer graphics** to generate fake car images instead of collecting real data.
- These images can look very realistic and help train models.

Problem: Overfitting to Limited Data

- Even if images look good to humans, they might only represent a **small subset** of all possible cars.

- Example:
  - A video game may only have **~20 car designs**.
  - It looks realistic to us, but in the real world, there are **many more types of cars**.
  - If we train only on these 20 cars → model may **overfit** and not generalize well.

Key Idea

- Artificial data synthesis **can work very well** (especially in speech recognition).
- But you must be careful:
  - Don’t generate data from a **small, limited set**
  - Make sure your data is **diverse enough**

What to Do if you think you have data mismatch

- Use **error analysis** to check differences between training and real-world data.
- Try to make training data **closer to real data (dev/test set)**.
- Use artificial data, but ensure **high diversity**.

Takeaway

- Artificial data = powerful
- But low diversity = risk of **overfitting**